MCP Explained

Model Context Protocol explained in depth for Engineers

1 The problem MCP solves

> **Without MCP, every LLM-to-tool connection is a hand-written integration, so 3 models and 5 tools cost 15 integrations.**

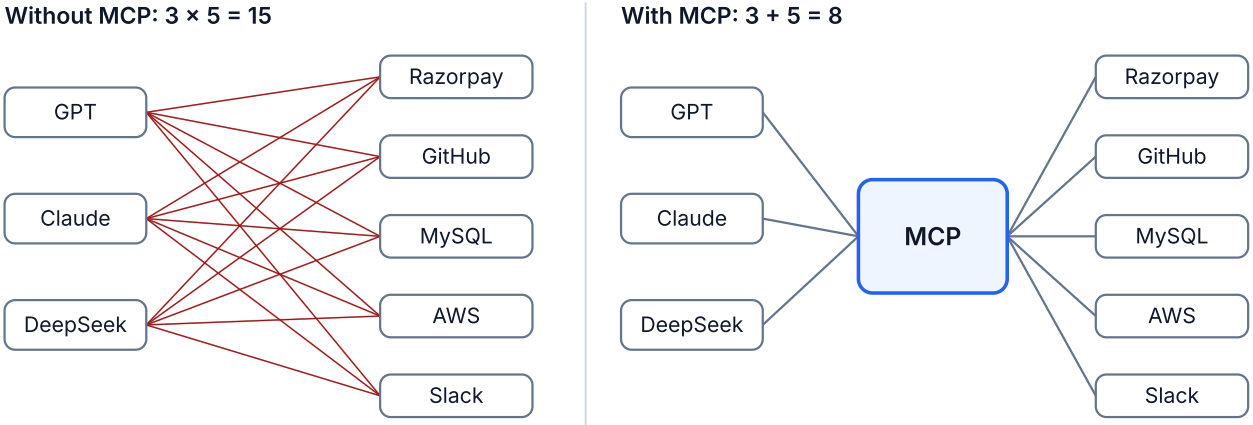
Running example: an AI customer support bot for an e-commerce site. "Where is my order?" needs MySQL; "Refund it" needs Razorpay and a Slack alert to the team.

For each tool, a custom integration means you:

1. Call that tool's specific API endpoint.
2. Parse its response with a custom parser.
3. Update the code and the parser, then redeploy, whenever the API changes.

This is the classic **N × M problem**: N models × M tools = N × M integrations. It is hard to build, hard to maintain, and breaks when you swap the model (GPT → DeepSeek) or the tool (MySQL → PostgreSQL).

MCP turns 15 custom integrations into 8 connections



Adding a model or a tool adds one connection, not one per pair.

2 What MCP is

> **MCP is a standard middleman between an AI app and its tools: write the integration once, and it survives model and tool changes.**

- **Analogy:** a USB-C hub. Without one, each device needs its own port on the laptop (custom logic per device). With one, every device plugs into the hub and the laptop needs a single USB-C port. MCP is that port.

- **Like HTTP, for AI:** HTTP is the protocol a browser uses to talk to web servers. MCP is the standard protocol any AI application uses to talk to any tool.
- **Built for models, not humans:** traditional APIs are designed for developers who read docs and hard-code calls. MCP is designed for AI models that discover tools and pick one at runtime.

What the name means

Word	Why
Model	Built specifically for AI models and AI apps, not for mobile or web apps.
Context	Its job is to bring the right external context to the model: tool definitions, database schemas, enterprise data.
Protocol	A set of rules for communication, the way HTTP is.

What MCP does not do: it adds no new capability beyond what the tool's own API allows. If GitHub doesn't allow deleting repos externally, MCP can't do it either. MCP only standardizes how the interaction happens.

3 Tools: the unit of work

> **In MCP, a tool is an executable function the AI can call to perform an action; it takes JSON input and returns a JSON result.**

Example: the GitHub MCP server exposes tools such as `create_repository`, `delete_repository`, `issue_read` and `issue_write`.

```
// input to create_repository
{ "name": "mcp-demo" }

// result
{ "id": 42, "status": "created" }
```

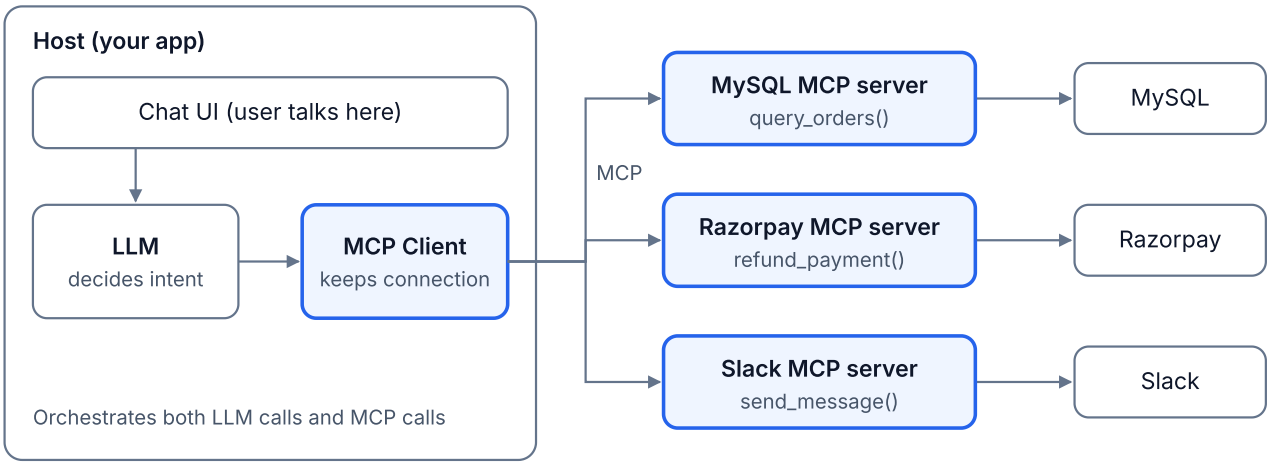
Tools are an abstraction layer. The LLM sees `create_repository`; it never needs to know the REST endpoint behind it (`POST /user/repos`). Endpoint changes are absorbed by the server, not by your app or your prompts.

4 Architecture: Host, Client, Server

> **MCP has three components: the Host runs the app and the LLM, the MCP Client inside it holds the connection, and each MCP Server wraps a tool.**

Component	What it is	Example
Host	The LLM-powered application users interact with. Custom code in any language or framework. Owns the client and orchestrates both LLM calls and MCP calls.	A chatbot backend; Codex or Claude Code while you code
MCP Client	Lives inside the host. Maintains a connection to an MCP server, sends requests to it, and gets context back for the host to use.	The support bot's MCP connector
MCP Server	A translator in front of a tool. Runs the command or fetches the data, then returns it in a format the AI can read. Technically, the program that serves context data.	A MySQL MCP server, the GitHub MCP server

The host owns the client; each MCP server fronts one tool and its keys



API keys and secrets live in the servers. The LLM only names a tool and reads the result.

The host holds the LLM and the client; the client reaches one MCP server per tool, and each server holds that tool's credentials.

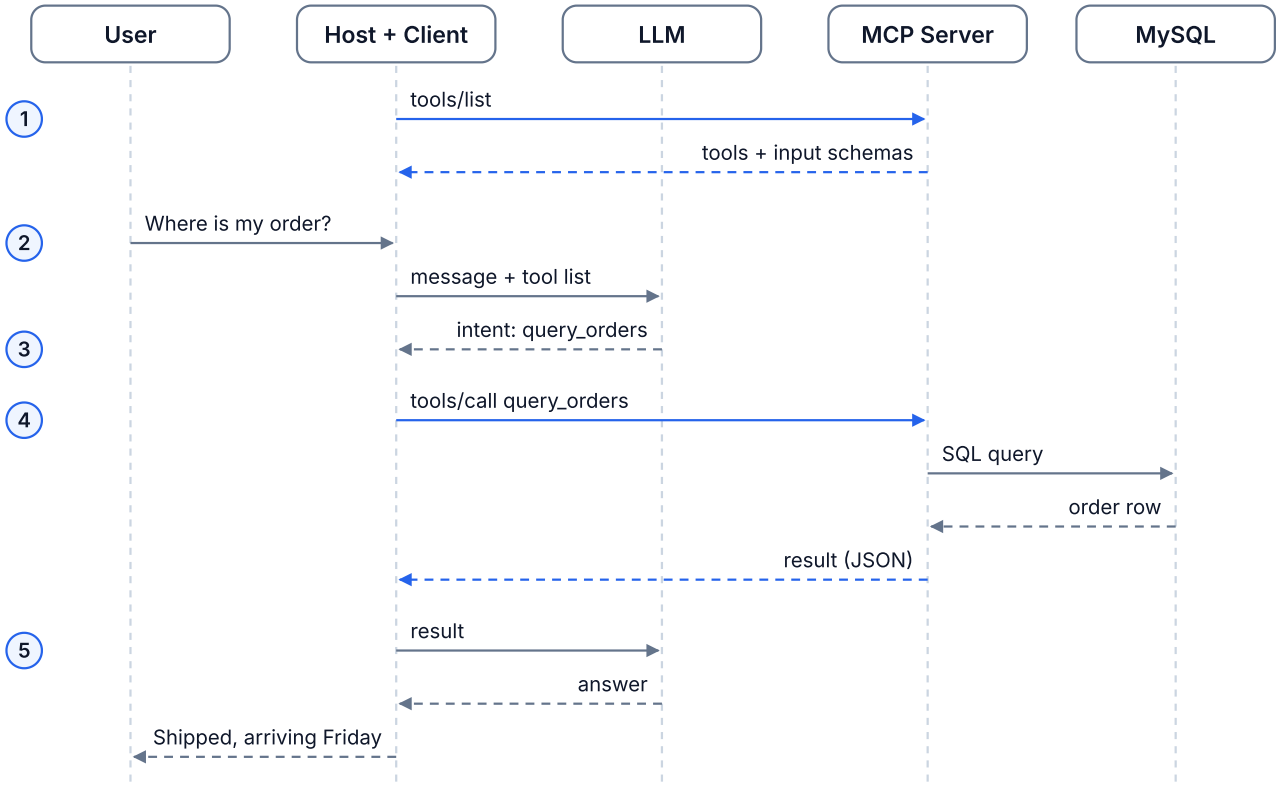
5 End-to-end: one support conversation

> **The LLM only decides; the host executes every tool call through MCP, and results stay in context for follow-ups.**

1. **Tool discovery.** At startup the client asks each MCP server what tools it has (`tools/list`). Each server replies with its tools and an input schema per tool. The host stores the list, so the LLM knows the tools without any hard-coding.
2. **User query.** The user asks "Where is my order #4821?" The host sends the message to the LLM along with the tool list.
3. **LLM decides.** The LLM states an intent, e.g. call `query_orders` with `order_id: 4821`. It does not execute anything itself.
4. **Host executes.** The host sends the tool request through the client to the MCP server. The server runs it against MySQL and returns the result.

5. **LLM responds.** The host forwards the result to the LLM, which writes the answer ("Shipped, arriving Friday").
6. **Conversation continues.** The user asks for a refund. The order and product details are already in context, so the LLM goes straight to Razorpay's `refund_payment` with no second database query.

The LLM only decides; the host makes every tool call through MCP



Step 6: on a follow-up refund, the order is already in context, so the LLM calls Razorpay refund_payment directly.

Blue arrows are MCP traffic; dashed arrows are replies. The LLM never talks to the server or the database directly.

6 Benefits

> **MCP turns N × M integrations into N + M, discovers tools at runtime, and keeps credentials away from the LLM.**

Benefit	Without MCP	With MCP
Integration count	N × M custom integrations (3 LLMs × 5 tools = 15)	N + M: each model connects to MCP once, each tool has one server (3 + 5 = 8)
Maintenance	Every API change means recode, update parser, redeploy	Changes are handled inside the MCP server
Discovery	Tools hard-coded into the app	The client asks servers for available tools and capabilities; the LLM picks at runtime
Switching LLMs	Rewrite integrations per model	Often a one-line change (the model name)

Benefit	Without MCP	With MCP
Security	5 services = 5 API keys, handed to the LLM if it connects directly	Keys, tokens and secrets live in the MCP server; the LLM only invokes tools and never sees them

7 Stateless MCP and scaling

> **MCP started out stateful and has moved toward being stateless: the server treats every request on its own, not as part of a session.**

- **Stateful:** requests belong to a session the server remembers, so a client must keep hitting the same server instance (session pinning).
- **Stateless:** each request carries what it needs, so any server instance can serve it.
- **Why it matters:** horizontal scaling gets simpler. Add server instances behind a load balancer and use round-robin, with no session pinning.

Open question: confirm against the current MCP spec version before relying on stateless behaviour in production.

8 Who builds MCP servers

> **Prefer the vendor's official server, fall back to open source, and build your own when you need control or custom logic.**

Type	Built by	Strengths	Watch out for
Enterprise (official)	The tool's company, e.g. GitHub's own MCP server	They know their API best and keep the server current	Only exists for some tools
Open source	Community, e.g. Anthropic's reference servers (file system, databases, web)	Good starting point when no official server exists	Can be limited for production; you must track updates
Custom	Your team	Easier to manage and secure; custom business logic; fits enterprise needs	You own building and maintaining it

One server per tool, or one for many? Either works. Aggregators like Zapier, Pipedream and Make expose thousands of tools through one MCP server. You can also build your own aggregator for internal tools when no existing server reaches the APIs you need.

9 Quick reference

Term	Meaning
Tool	Executable function on an MCP server; JSON in, JSON out
`tools/list`	Discovery request: "what tools do you have?" Returns tools with input schemas
`tools/call`	Invokes one tool with arguments
Intent	The LLM's decision about which tool to call with which arguments; the host executes it
Host	Your LLM app; owns the client, orchestrates LLM and MCP calls
MCP Client	Connection from the host to one MCP server
MCP Server	Wraps a tool or API, holds its credentials, serves context
Aggregator	One MCP server exposing many tools (Zapier, Pipedream, Make, or your own)

Beyond the video: per the MCP spec, messages are JSON-RPC 2.0, the common transports are stdio (local) and Streamable HTTP (remote), and servers can also expose resources and prompts alongside tools.

Keep Learning & Keep Exploring!